



Feature Engineering

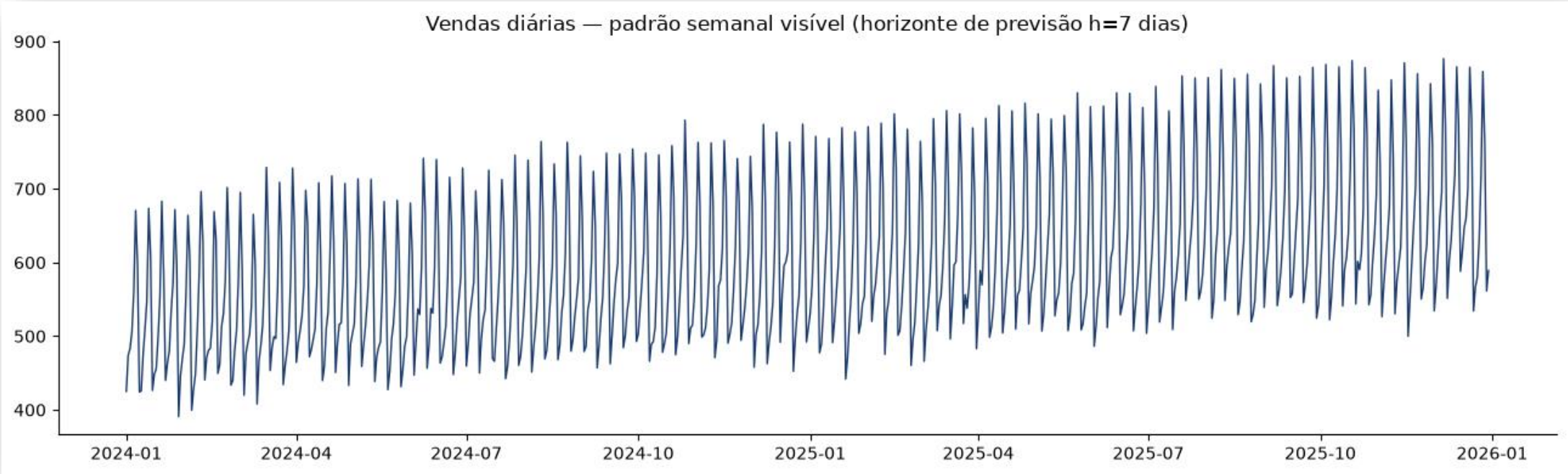
Daniel de Freitas

Feature Engineering para Séries Temporais



- **AR, SARIMAX e Holt-Winters** trabalham diretamente com a série temporal.
- **Modelos de Machine Learning** recebem dados organizados em tabelas.
- **Feature Engineering** é o processo de criar, transformar e selecionar variáveis a partir dos dados brutos para facilitar o aprendizado do modelo.
- **Feature Engineering para séries temporais** transforma padrões temporais em colunas explicativas.
- Lags, janelas móveis e calendário fornecem “memória” ao modelo.
- **O objetivo é preparar os dados para os modelos de ML.**





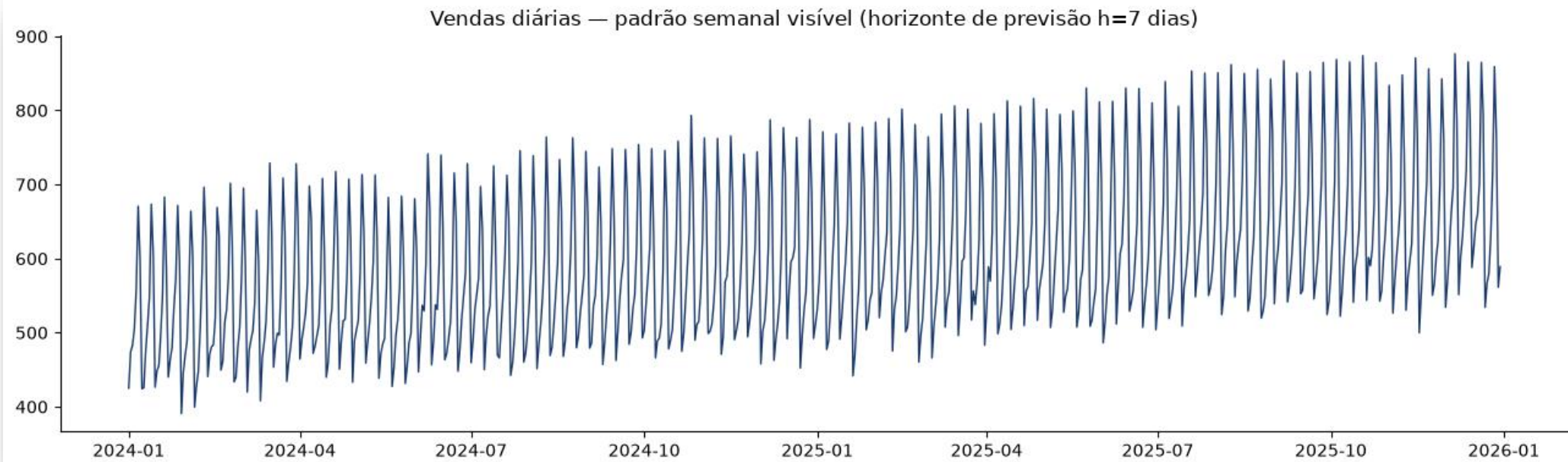
Uma rede varejista precisa solicitar diariamente ao fornecedor, com sete dias de antecedência, a quantidade de produtos que será entregue em cada loja. Os pedidos dos próximos seis dias já foram realizados anteriormente. Com base nas vendas conhecidas até hoje, quantas unidades deverão ser solicitadas para entrega exatamente daqui a sete dias?

Série



```
h = 7 # horizonte de previsão - 1 semana à frente

fig, ax = plt.subplots(figsize=(13, 4))
ax.plot(df.index, df['vendas'], color='#1a3a6e', linewidth=1)
ax.set_title(f'Vendas diárias: padrão semanal visível (horizonte de previsão h={h} dias)')
plt.tight_layout()
plt.show()
```



Lag Features: criando memória para o modelo



- Uma lag feature representa um valor anterior da própria série.
- **lag_k** corresponde ao valor observado k períodos atrás.
- Como $h=7$, o menor lag seguro é `lag\_7`
- **lag_1** representa o período anterior; **lag_7**, uma semana atrás.
- **Os lags ajudam o modelo a capturar autocorrelação e sazonalidade.**
- A disponibilidade do lag depende da data em que a previsão é realizada.

```
feat = pd.DataFrame(index=df.index)
feat['y'] = df['vendas']

for lag in [7, 14, 21, 28]:
    feat[f'lag_{lag}'] = df['vendas'].shift(lag)

feat[['y', 'lag_7', 'lag_14', 'lag_21', 'lag_28']].tail(8)
```

✓ 0.0s × given + Tag

	y	lag_7	lag_14	lag_21	lag_28
data					
2025-12-23	566.567477	614.061580	602.338450	568.715816	564.557060
2025-12-24	579.241861	646.427938	628.968042	612.923382	600.896758
2025-12-25	628.235783	659.777742	673.425154	661.084290	624.275162
2025-12-26	713.892827	701.306068	719.027643	695.444197	700.156052
2025-12-27	859.147324	864.836124	865.504731	876.544684	842.562613
2025-12-28	763.263084	792.850582	788.830754	785.561982	762.063070
2025-12-29	561.139707	534.098081	587.845175	551.271173	534.157213
2025-12-30	589.050229	566.567477	614.061580	602.338450	568.715816

Lag Features: criando memória para o modelo



- A chave é esta: a linha de 30/12 representa o dia que queremos prever, mas a previsão seria feita em 23/12, sete dias antes.
- O `shift(7)` simplesmente pega o valor de 23/12 e o desloca para a linha de 30/12;
- **Lags intermediários** (8,9,10,11,12,13,15,16...) **enriquecem o histórico recente e podem ajudar o modelo a inferir mudanças de nível e tendência**, mas não representam a tendência de forma explícita.
- **Expanda essa ideia e considere todos os lags intermediários válidos até o 28**

```
feat = pd.DataFrame(index=df.index)
feat['y'] = df['vendas']

for lag in [7, 14, 21, 28]:
    feat[f'lag_{lag}'] = df['vendas'].shift(lag)

feat[['y', 'lag_7', 'lag_14', 'lag_21', 'lag_28']].tail(8)
```

✓ 0.0s × given + Tag

	y	lag_7	lag_14	lag_21	lag_28
data					
2025-12-23	566.567477	614.061580	602.338450	568.715816	564.557060
2025-12-24	579.241861	646.427938	628.968042	612.923382	600.896758
2025-12-25	628.235783	659.777742	673.425154	661.084290	624.275162
2025-12-26	713.892827	701.306068	719.027643	695.444197	700.156052
2025-12-27	859.147324	864.836124	865.504731	876.544684	842.562613
2025-12-28	763.263084	792.850582	788.830754	785.561982	762.063070
2025-12-29	561.139707	534.098081	587.845175	551.271173	534.157213
2025-12-30	589.050229	566.567477	614.061580	602.338450	568.715816

Rolling Features: resumindo o histórico



- Rolling features calculam estatísticas sobre uma janela de valores anteriores.
- A **média móvel** representa o nível recente da série.
- O **desvio padrão móvel** representa a variação ou volatilidade recente.
- **Mínimo** e **máximo** ajudam a identificar períodos de valores extremos.
- O **shift(h)** garante que a janela termine na última informação disponível.
- As **estatísticas resumem vários valores históricos em poucas features mais estáveis** e menos sensíveis a ruídos pontuais.
- Elas **permitem que o modelo reconheça mudanças de nível e volatilidade sem precisar interpretar diretamente a sequência temporal**.

```
feat['roll_mean_7'] = df['vendas'].shift(h).rolling(7).mean()  
feat['roll_std_7'] = df['vendas'].shift(h).rolling(7).std()  
feat['roll_mean_28'] = df['vendas'].shift(h).rolling(28).mean()
```

```
feat[['y', 'roll_mean_7', 'roll_std_7', 'roll_mean_28']].tail(8)
```

✓ 0.0s × given + Tag

	y	roll_mean_7	roll_std_7	roll_mean_28
data				
2025-12-23	566.567477	696.809011	101.068765	677.948612
2025-12-24	579.241861	699.303282	99.315723	679.549861
2025-12-25	628.235783	697.353652	100.039709	679.801360
2025-12-26	713.892827	694.821998	99.623170	679.731486
2025-12-27	859.147324	694.726483	99.432388	680.040067
2025-12-28	763.263084	695.300744	100.075984	681.277076
2025-12-29	561.139707	687.622588	111.144153	680.696840
2025-12-30	589.050229	680.837716	117.642830	680.768641

Features de Calendário



- São variáveis extraídas diretamente da data de cada observação.
- Exemplos: dia da semana, mês, trimestre, feriado e fim de semana.
- Permitem que o modelo reconheça padrões relacionados ao calendário.
- São conhecidas antecipadamente, inclusive para datas futuras.
- Complementam lags e rolling features quando o calendário influencia a demanda.

```
feat['dow'] = feat.index.dayofweek # 0=segunda ... 6=domingo
feat['month'] = feat.index.month
feat['is_weekend'] = (feat['dow'] >= 5).astype(int)

feat[['dow', 'month', 'is_weekend']].tail(8)
```

✓ 0.0s × given + Tag

	dow	month	is_weekend
data			
2025-12-23	1	12	0
2025-12-24	2	12	0
2025-12-25	3	12	0
2025-12-26	4	12	0
2025-12-27	5	12	1
2025-12-28	6	12	1
2025-12-29	0	12	0
2025-12-30	1	12	0

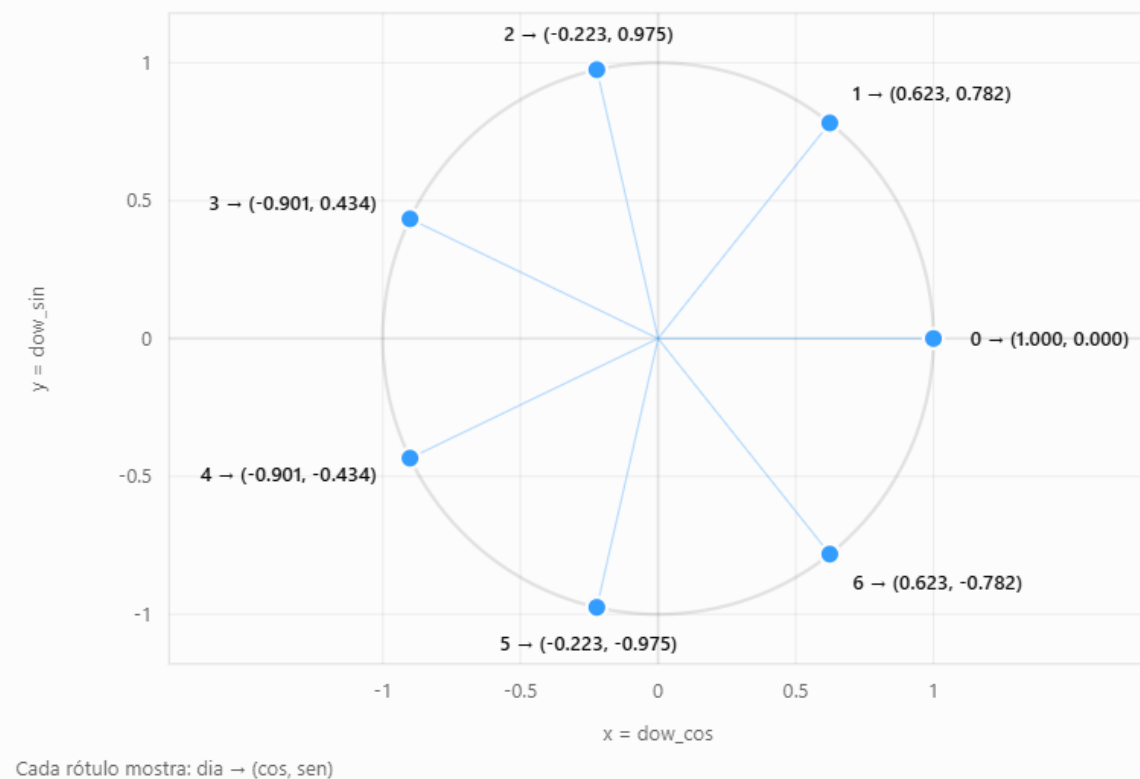
Encoding Cíclico



- Algumas variáveis de calendário possuem comportamento circular.
- Domingo e segunda-feira são próximos, apesar de 6 e 0 parecerem distantes.
- Seno e cosseno transformam os valores em posições sobre um círculo.
- O par $\sin + \cos$ identifica corretamente cada posição do ciclo.
- É especialmente útil para modelos lineares ou baseados em distância.

Encoding Cíclico

- Algumas variáveis de calendário possuem comportamento circular.
- Domingo e segunda-feira são próximos, apesar de 6 e 0 parecerem distantes.
- Seno e cosseno transformam os valores em posições sobre um círculo.
- O par $\sin + \cos$ identifica corretamente cada posição do ciclo.
- É especialmente útil para modelos lineares ou baseados em distância.



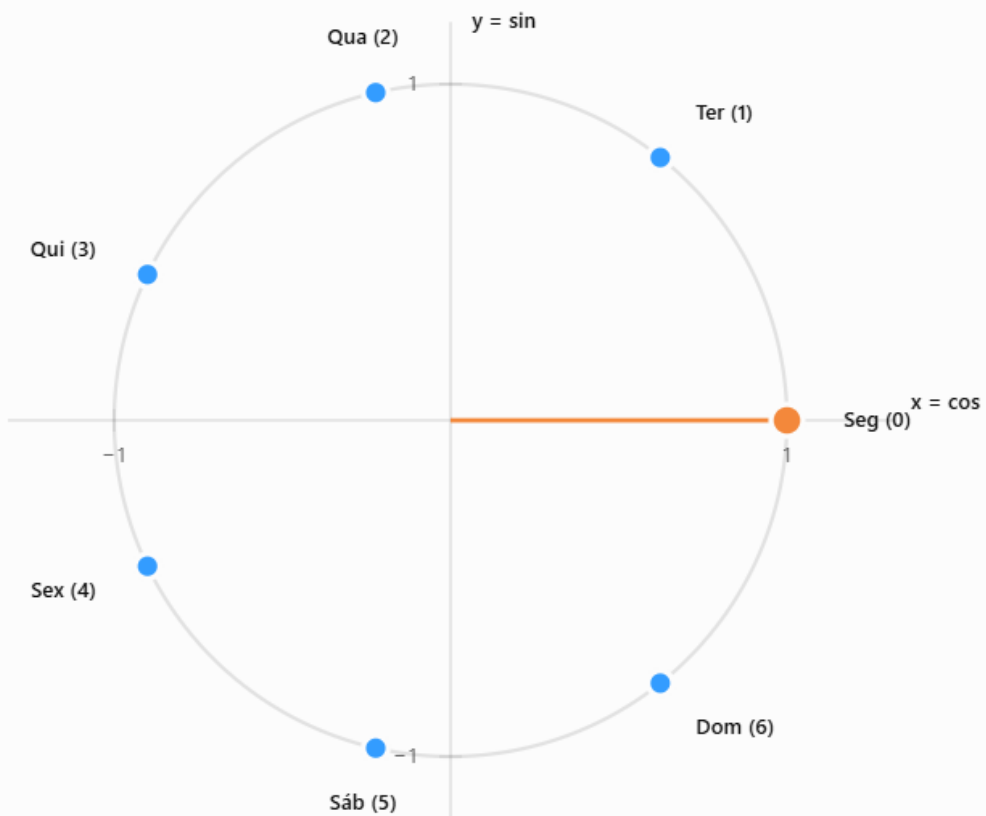
Encoding Cíclico



Dias da semana no círculo unitário

No gráfico: $(x, y) = (\text{cosseno}, \text{seno})$

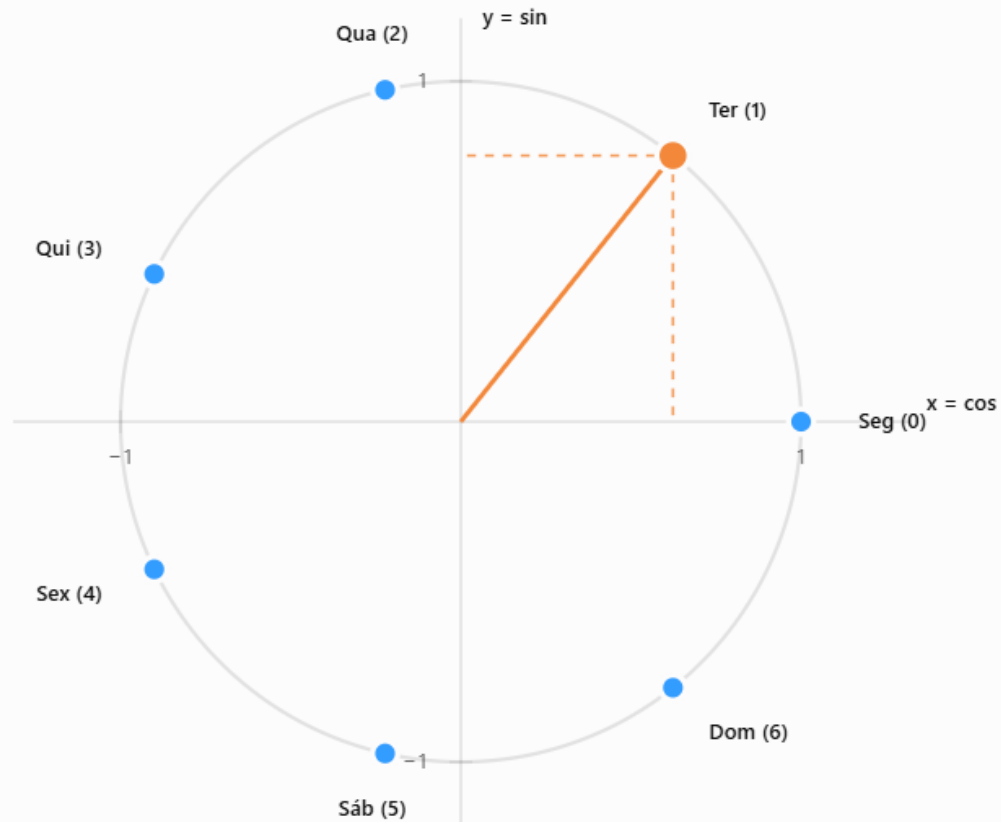
Seg Ter Qua Qui Sex Sáb Dom



Dias da semana no círculo unitário

No gráfico: $(x, y) = (\text{cosseno}, \text{seno})$

Seg Ter Qua Qui Sex Sáb Dom



Encoding Cíclico



Encoding Cíclico

- Algumas variáveis de calendário possuem comportamento circular.
- Domingo e segunda-feira são próximos, apesar de 6 e 0 parecerem distantes.
- Seno e cosseno transformam os valores em posições sobre um círculo.
- O par sin + cos identifica corretamente cada posição do ciclo.
- É especialmente útil para modelos lineares ou baseados em distância.

```
feat['dow_sin'] = np.sin(2 * np.pi * feat['dow'] / 7)  
feat['dow_cos'] = np.cos(2 * np.pi * feat['dow'] / 7)  
feat['month_sin'] = np.sin(2 * np.pi * feat['month'] / 12)  
feat['month_cos'] = np.cos(2 * np.pi * feat['month'] / 12)
```

```
feat[['dow', 'dow_sin', 'dow_cos', 'month', 'month_sin', 'month_cos']].tail(8)
```

✓ 0.0s × solution + Tag

	dow	dow_sin	dow_cos	month	month_sin	month_cos
data						
2025-12-23	1	0.781831	0.623490	12	-2.449294e-16	1.0
2025-12-24	2	0.974928	-0.222521	12	-2.449294e-16	1.0
2025-12-25	3	0.433884	-0.900969	12	-2.449294e-16	1.0
2025-12-26	4	-0.433884	-0.900969	12	-2.449294e-16	1.0
2025-12-27	5	-0.974928	-0.222521	12	-2.449294e-16	1.0
2025-12-28	6	-0.781831	0.623490	12	-2.449294e-16	1.0
2025-12-29	0	0.000000	1.000000	12	-2.449294e-16	1.0
2025-12-30	1	0.781831	0.623490	12	-2.449294e-16	1.0

Construção da tabela final



- Lags e janelas móveis criam valores **NaN** no início da série.
- Isso acontece porque ainda não existe histórico suficiente para calcular algumas features.
- A maior exigência de histórico determina quantas linhas iniciais serão perdidas.
- Linhas com features incompletas não podem ser utilizadas diretamente no treinamento.
- O `dropna()` gera a tabela final, mas reduz a quantidade de dados disponíveis.

```
linhas_antes = len(feat)
feat_clean = feat.dropna()
linhas_depois = len(feat_clean)

print(f"Linhas antes do dropna: {linhas_antes}")
print(f"Linhas depois do dropna: {linhas_depois}")
print(f"Perdas no aquecimento das janelas: {linhas_antes - linhas_depois} "
      f"f'(({linhas_antes - linhas_depois) / linhas_antes:.1%} da série)")
print("O maior lag (28) somado ao deslocamento por h (7) determina esse custo: "
      "quanto mais longa a janela, mais linhas iniciais ficam sem features completas.")
```

✓ 0.0s × given + Tag

Python ▾

```
Linhas antes do dropna: 730
Linhas depois do dropna: 696
Perdas no aquecimento das janelas: 34 (4.7% da série)
O maior lag (28) somado ao deslocamento por h (7) determina esse custo: quanto mais longa a janela,
```

Divisão treino-teste

- O conjunto de treino contém as observações mais antigas da série.
- O conjunto de teste contém as observações mais recentes.
- A ordem cronológica deve ser preservada, sem embaralhamento.
- O teste simula previsões para um período ainda não visto pelo modelo.
- Neste exemplo, o modelo fica fixo e produz previsões diárias para D+7.

```
test_size = 60
train = feat_clean.iloc[:-test_size]
test = feat_clean.iloc[-test_size:]
feature_cols = [c for c in feat_clean.columns if c != 'y']

print(f"Treino: {len(train)} dias | Teste: {len(test)} dias")
print(f"Features construídas ({len(feature_cols)}): {feature_cols}")
```

✓ 0.0s × given + Tag

Pyth

Treino: 636 dias | Teste: 60 dias

Features construídas (14): ['lag_7', 'lag_14', 'lag_21', 'lag_28', 'roll_mean_7', 'roll_std_7',

Modelo de ML: RandomForest



- X contém as features criadas a partir da série temporal.
- y contém o valor de vendas que o modelo deverá prever.
- O Random Forest combina várias árvores de decisão.
- O método fit() aprende as relações entre as features e o alvo.

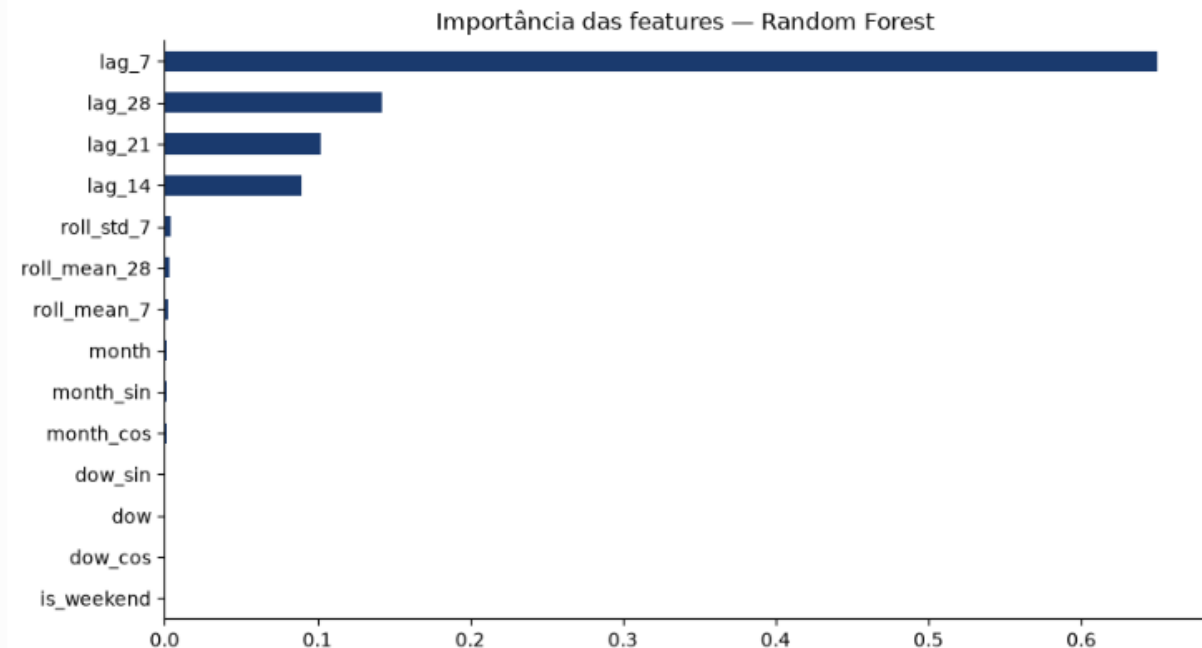
```
modelo = RandomForestRegressor(n_estimators=200, random_state=42)
modelo.fit(train[feature_cols], train['y'])
previsoes = modelo.predict(test[feature_cols])
mae = mean_absolute_error(test['y'], previsoes)
print(f"MAE (Random Forest sobre as features construídas): {mae:.2f}")

importancias = pd.Series(modelo.feature_importances_, index=feature_cols).sort_values()
fig, ax = plt.subplots(figsize=(9, 5))
importancias.plot(kind='barh', ax=ax, color='#1a3a6e')
ax.set_title('Importância das features - Random Forest')
plt.tight_layout()
plt.show()

print(importancias.sort_values(ascending=False).round(4))
```

Importância das Features

- A importância indica quanto cada feature foi utilizada pelas árvores do modelo.
- Valores maiores representam maior contribuição para as decisões da Random Forest.
- lag_7 foi a feature mais importante nesta série.
- Features correlacionadas podem dividir a importância entre si.
- Importância não significa causalidade e pode mudar em outra série ou modelo.



```
lag_7          0.6495
lag_28         0.1427
lag_21         0.1026
lag_14         0.0901
roll_std_7     0.0038
roll_mean_28   0.0028
roll_mean_7    0.0023
month          0.0018
month_sin      0.0017
month_cos      0.0016
dow_sin        0.0004
dow            0.0004
dow_cos        0.0003
is_weekend     0.0001
dtype: float64
```

